2400-DS1AL # Algorithms for Data Science

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 1: Introduction

How to Learn?

- Literature:
  - A good reference on algorithms: *Introduction to Algorithms* by Cormen, Leiserson, and Rivest, Stein. A rather big book, includes both the easier subjects (such as those we will be talking about in this course) as well as some of the more advanced ones.
  - Good alternative: *Algorithm Design* by John Kleinberg and Eva Tardos. Also good seems to be *Introduction to Algorithms: A Creative Approach* by Udi Manber.
  - Easier to grasp for beginners: *Data Structures and Algorithms Made Easy* by Narasimha Karumanchi, and *Grokking Algorithms: An illustrated guide for programmers and other curious people* by Aditya Bhargava. Though I haven't read these two books, I have seen good opinions about them.
- Internet resources, for example
  - free online courses on smurf.mimuw.edu.pl (in Polish) or on ocw.mit.edu (as the introduction to algorithms)
  - tutorials to learn/recall C++ or Python as in

- A good way to get practical experience is to look into competitive programming. This is a kind of a mental sport where one gets an algorithmic problem, has to create an algorithm, and implement it in the chosen programming language. Competitive programming sites allow one to submit your solution and have it tested automatically. One can compete against others (usually under time limit) or use this for educational purposes. Some sites:
  - HackerRank—many educational problems (you can actually learn Python and C++ in this way)
  - CodeForces—currently the biggest community

What Programming Languages We Will Be Using?

- Pseudocode—when we want to present the idea of an algorithm quickly without giving the implementation details
- **Python** and **C++**, a comparison:
  - Python is very popular in Data Science and Machine Learning (notable, ChatGPT has been written in Python), C++ is very popular among algorithmists (statistics)
  - It is quite easy to create very efficient programs in C++—computations requiring high performance may be possible in Python, but they are usually done with an external library (probably written in C++ or a similar language).
  - Python is believed to be very easy, while C++ is commonly believed to be hard and complex—however, there are people who think otherwise.
  - The easiest ways to use C++ on Windows are to install Code::Blocks, or to use an online compiler such as ideone.
  - C++ is an evolving language: it started when Bjarne Stroustrup added object oriented programming to C (a very low-level language according to the modern standards); after that, templates, and the *standard library* have been added. There were major new additions in 2014, 2017, 2020 and planned in 2023; these additions allow doing things much easier. Unfortunately, it is hard to find a good tutorial of modern C++—many of them concentrate too much on features which are not that useful for us, such as pointers (a low level and rather difficult concept which was essential in C, but can be mostly avoided in modern C++) or object oriented programming (useful for creating some kinds of software, but not really relevant for us). Learn C++ is a popular tutorial, but it also has these problems.
  - See below for a more detailed comparison.

Example Problem: "Guess The Number" Puzzle

I have a secret integer number x from 0 to 1000. You can ask questions of form "is x≤...?" How can you find the value of x?

The idea of the solution: first, ask whether x≤500. After we receive the answer, we either know that 0≤x≤500, or that 501≤ x≤1000. We ask about the

middle number of the new interval, again splitting the interval in half. After about 10 questions we know the exact value of `x` .

This algorithm can be described by the following fragment of a C++ program:

```
int l = 0;
int r = 1000;
// l (as left) and r (as right) denote our current bounds: we know that l <= x <= r
while (l < r) {
  // we will ask about the number in the middle
  int m = (l + r) / 2;
  // note: this is an integer division (e.g., for l=0, r=1 we get m=1/2=0)
  if (x <= m)
    r = m;
  else
    l = m + 1;
}
// Now l = x = r
```

After the execution of this fragment, both variables `l` and `r` will be equal to `x`.

Although the idea of the algorithm is very simple, it is not that straightforward to implement it precisely enough so that it can be executed by the computer. For example, if one writes `l=m` instead of `l=m+1`, the algorithm will not be working correctly: suppose that, at some point, `l=0`, `r=1`, and `m=1`. We compute `m=0`, and since the condition `x<=m` is not satisfied, we set `l` to `m`, which does not change anything—thus, we will keep asking about `x<=0` forever!

How to check that the program is correct then?

- Our loop has a so-called *invariant*: at each moment we have `l<=x<=r`.
  - First, we check whether the invariant is true when we enter the loop for the first time. As we know that `0<=x<=1000` and `l` and `r` have exactly these values, we see that it is true.
  - Next, we check whether the invariant is true after a single iteration of the while loop under the assumption that it is true at the beginning of that iteration. In other words, assuming that `l<=x<=r` holds at the beginning of an iteration, we check whether `l<=x<=r` also holds at the end of that iteration.

For this, we have to check both cases of the if statement: `x<=m` and the else part `x>m`. In the first case (`x<=m`), we set `r=m` which implies that `l<=x<=r` still holds. Similarly in the second case (`x>m`), after setting `l=m+1`, we still have `l<=x<=r`.

- By combining our two observations above, we conclude that the invariant is true at the end of the first iteration (since it was true at the beginning), and, hence, it is also true at the end of the second iteration, at the end of the third iteration, ..., at the end of any iteration including the last one. Thus, the invariant is also true after we exit the while loop. At this point, we know that `l<=x<=r` is true (the invariant) and that `l>=r` holds (the loop is left only if its condition `l<r` is violated), therefore both `l` and `r` must be equal to `x`.

- Our reasoning shows that the program gives the correct answer if the program terminates! To show that the program always ends, we can, for instance, consider the difference `r-l`. This value is an integer which gets smaller and smaller with each iteration (we have to check the two cases again), and never drops below `0`, so the program will eventually have to finish.

Application

The algorithm discussed in the last section is known as *binary search*. It has been motivated by a *puzzle* to show that algorithmics is not necessarily about computers. In fact, algorithms are useful in general, and they find applications also in areas unrelated to programming. Nonetheless, the very fundamental algorithm *binary search* has a lot of applications especially in programming.

For example, suppose we have an array of integers, i.e., `a[0], a[1], ..., a[n-1]`, and we know that it is nondecreasing. Given a value `v`, we want to find the smallest array position (also called index) `i` such that `a[i] ≥ v`; in case that all the integers in our array are smaller than `v`, we want to return the nonexisting position `n`. How to accomplish this task?

We use the same general idea as in the puzzle, but now instead of checking whether `x <= m`, we check whether `a[m] >= v`. We also start with `r=n` instead of `r=1000`.

For the interested reader, it is here worth to note that if we started with `r=n-1` instead of `r=n`, the output would be also correct with one exception: if all values in the array are smaller than `v`, the output would be wrongly `n-1` instead of `n`. To prevent this issue from happening, we need to start with `r=n`. One might be now worried that our program could try to read the value `a[n]` which is undefined. However, binary search reads only the value `a[m]`, and we know that `l≤m<r` as `m` is the middle point between `l` and `r` rounded down(!). Hence, `m<n` and `a[n]` will be never accessed.

We prove the correctness of our program in exactly the same way except that the *invariant* now says that "`l<=i<=r` where `i` is the position we look for as defined above." Alternatively, we can also use the following invariant: "`a[j]<v` holds for each index `j` smaller than `l`, and `a[j]>=v` for each index `j` greater or equal to `r`."

The Complete Program

Below is an example of a complete C++ program for the application above:

```
#include <iostream>
#include <vector>

// We separate our binary search as a function. For the array 'a', we use the type std::vector<int> from
// the standard library.

int bin_search(const std::vector<int>& a, int v) {

  // The keyword 'const' makes the vector 'a' read-only for the function.

  // The & sign denotes that 'a' is a so-called "reference",
  // which means that 'a' is identical with the vector passed as an argument to the function.
  // If & is omitted, the function makes its own copy of the vector given to it,
  // which takes lots of time. Thus it is important not to forget the & sign!
  // For simple variable types as 'int', there is no speedup using a reference.


  int l = 0;
  int r = a.size();
  while (l < r) {
    int m = (l + r) / 2;
    if(a[m] >= v)
      r = m;
    else
      l = m + 1;
  }

  return r;
}


int main() {
  // we construct an example array 'a':
  std::vector<int> a;
  for (int i = 0; i < 1000; i++)
    a.push_back(i * i);
```

```
  while (true) {
    int v;
    std::cin >> v;
    std::cout << "The answer is: " << bin_search(a, v) << "\n";
  }

  return 0;
}
```

And a Python version:

```
def bin_search(a, v):
  l = 0
  r = len(a)
  while l < r:
    # In Python '//' is used for integer division
    # whereas '/' is used for normal division.
    m = (l + r) // 2
    if a[m] >= v:
      r = m
    else:
      l = m + 1
  return r

a = [i * i for i in range(100)]

while 1:
  v = int(input())
  print("The answer is: " + str(bin_search(a,v)))
```

The important differences between the C++ and Python versions:

- C++ is statically typed, while Python is dynamically typed. This means that the C programmer has to specify the type for each variable, which is not necessary in Python. This makes Python programs shorter; however, in Python, if you make a type error (for example, you forget to use the `int()` or `str()` functions which convert strings to integers and back), you will not know about this until you actually run the program

and it breaks when it reaches the offending part. In C++, since the compiler knows all the types, it reports an error right away during the compilation. Another advantage of C++ is that we do not have to store the type of the variable in the memory, nor to check it during the run time—i.e., a C++ program uses both the memory and processor much more efficiently. (In many cases it is possible to declare a variable as `auto` to make the compiler determine the type by itself, and use *templates* to create functions which work on many different types—without losing the advantages of static type checking mentioned above.)

- The C++ programmer decides whether to pass `a` as a reference or not, while the Python programmer just says `a`; Python does the correct thing by default in this particular case, but sometimes you might want a reference when Python does not allow it, or be surprised when Python does a reference when you want copy (try to run `x = [0]; y = [x for _ in range(5)]; y[0][0] = 1; print(y);` see here this and another example with explanations; for comparison, here or here the corresponding code in C++). Declaring `a` as a `const` in C++ helps the programmer (by telling that this variable is an input that is not changed) and the compiler (to generate more efficient code).

- `std::` is a *namespace* containing all the types and functions from the standard library; this is for backwards compatibility—if a C programmer called their own type `vector` while `std::vector` did not yet exist, their program will still work. (Most C programs will still work in modern C++, while Python 2 programs are likely not to work in Python 3.) It is possible to write `using namespace std;` to avoid the necessity of writing `std::` each time, although, we could lose backwards compatibility that way, when new things are added to `std`.

- In C++, the structure of the program (what is inside the `while` loop, and what is not) comes from the braces. It is a good style to use indentation, but it is not strictly necessary. In Python, there are no braces, and the structure comes from indentation.

- In C++ you need a bit of more code (e.g. the `main` function); on the other hand, this part will be the same in mostly every program.

- The syntax of the `for` loop and input/output might be a bit weird—C++11 allows nicer constructs, but they are not yet in the standard library (probably C++ programmers are too used to this to care). It is possible to make it nicer if you write or use a library such as easy.cpp of Eryk Kopczyński:

```
#include "easy.cpp"
using namespace easy;

int bin_search(const vector<int>& a, int v) {
  int l = 0;
  int r = a.size();
  while (l < r) {
    int m = (l + r) / 2;
    if (a[m] >= v)
      r = m;
    else
      l = m + 1;
    }
  return r;
}

int main() {
  vector<int> a;
  for (int i : ints(1000))
    a.push_back(i * i);

  while (true) {
    int v;
    read(v);
    writeln("The answer is: ", bin_search(a,v));
  }

  return 0;
}
```

The Importance of Using Efficient Algorithms

A simpler algorithm for the last problem is *linear search*, where you simply walk through the array from the beginning to the end until you find the first (and, hence, smallest) position `i` with `a[i] ≥ v`.

```
int i = 0;
while (i < n && a[i] < v) i++;
```

Note that if the array does not contain such a position (that is, all values are smaller than v), then `i` will be correctly set to n at the end.

How important is it to use a faster algorithm, compared to using a faster computer, or a faster programming language? Consider the case of n=1,000,000,000 (which is less, e.g., than the estimated number of websites in the year 2024). In the '80s, in BASIC, the interpreter could do about 1000 simple operations per second. Therefore, binary search (30 iterations) would take about 50 ms, while linear search would take about 12 days. On the other hand, C++ on a modern computer can do about 1000000000 operations per second—thus, linear search will take about 1 second, while the binary search takes about 50 nanoseconds. (By making n larger, the difference gets more and more impressive. For example., if we multiply n by 1 mio, then a modern computer needs about 12 days with linear search, whereas for the historic computer with binary search there is no noticeable increase in the running time, i.e., it is still about 50 ms.)

As this example shows—using a faster algorithm is more important than using a faster computer or a faster programming language. An efficient algorithm will be faster than a bad one, even when given a significant technological disadvantage! And even if answering one query per second does not sound that bad, we would usually use binary search as a part of a bigger system, which needs to find a value in an array many times. Coders with no knowledge of algorithmics would just implement the linear search algorithm (thinking it should be fast enough), to only find out that their program works very slowly, and then get amazed when told about binary search (even though it is a very simple algorithm).

In binary search, the number of iterations of the while loop equals the binary logarithm of n, while in linear search the number of iterations is equal to n in the worst case. To get the actual running time in some time measure, we have to multiply the number of iterations by a constant that depends on the used technology and on how many elementary operations we do in each iteration of the loop—for example, if we considered the case of `x=m` as a third case separately, the number of operations per iteration would be different. While we could save time by changing this, or by improving our technology, the difference between `log(n)` and n is the most significant difference—therefore, in our course, our foremost concern will be to make the *order* of the running time as small as possible. This will be explained in more detail in the next lectures.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:13

◄ Recording Przejdź do... Lecture 2 ►

Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)

2400-DS1AL#2025L#277553